

Programmierung und Deskriptive Statistik

BSc Psychologie WiSe 2026/27

Belinda Fleischmann

	Datum	Format	Thema
1	Mi, 14.10.	Seminar	(1) Grundbegriffe der Informatik
2	**Mi, 21.10.**	**Seminar**	**(2) Arithmetik und Variablen**
3	Mi, 28.10.	Übung	(2) Arithmetik und Variablen
4	Mi, 04.11.	Seminar	(3) Vektoren und Matrizen, (4) Listen und Dataframes
5	Mi, 11.11.	Übung	(3) Vektoren und Matrizen
6	Mi, 18.11.	Übung	(4) Listen und Dataframes
7	Mi, 25.11.	Seminar	(5) Datenmanagement
8	Mi, 02.12.	Übung	(5) Datenmanagement
	Mo, 07.12.	Abgabe	*Individualleistung (1/2)*
9	Mi, 09.12.	Seminar	(6) Strukturiertes Progr.: Kontrollfluss, Debugging
10	Mi, 16.12.	Seminar	(7) Häufigkeitsverteilungen
11	Mi, 13.01.	Übung	(7) Häufigkeitsverteilungen
12	Mi, 20.01.	Seminar	(8) Maße der zentralen Tendenz und Datenvariabilität
13	Mi, 27.01.	Übung	(8) Maße der zentralen Tendenz und Datenvariabilität
	Mo, 01.02.	Abgabe	*Individualleistung (2/2)*

(2) Arithmetik und Variablen

Getting started

Arithmetik, Logik und Präzedenz

Variablen

Datenstrukturen

Getting started

Arithmetik, Logik und Präzedenz

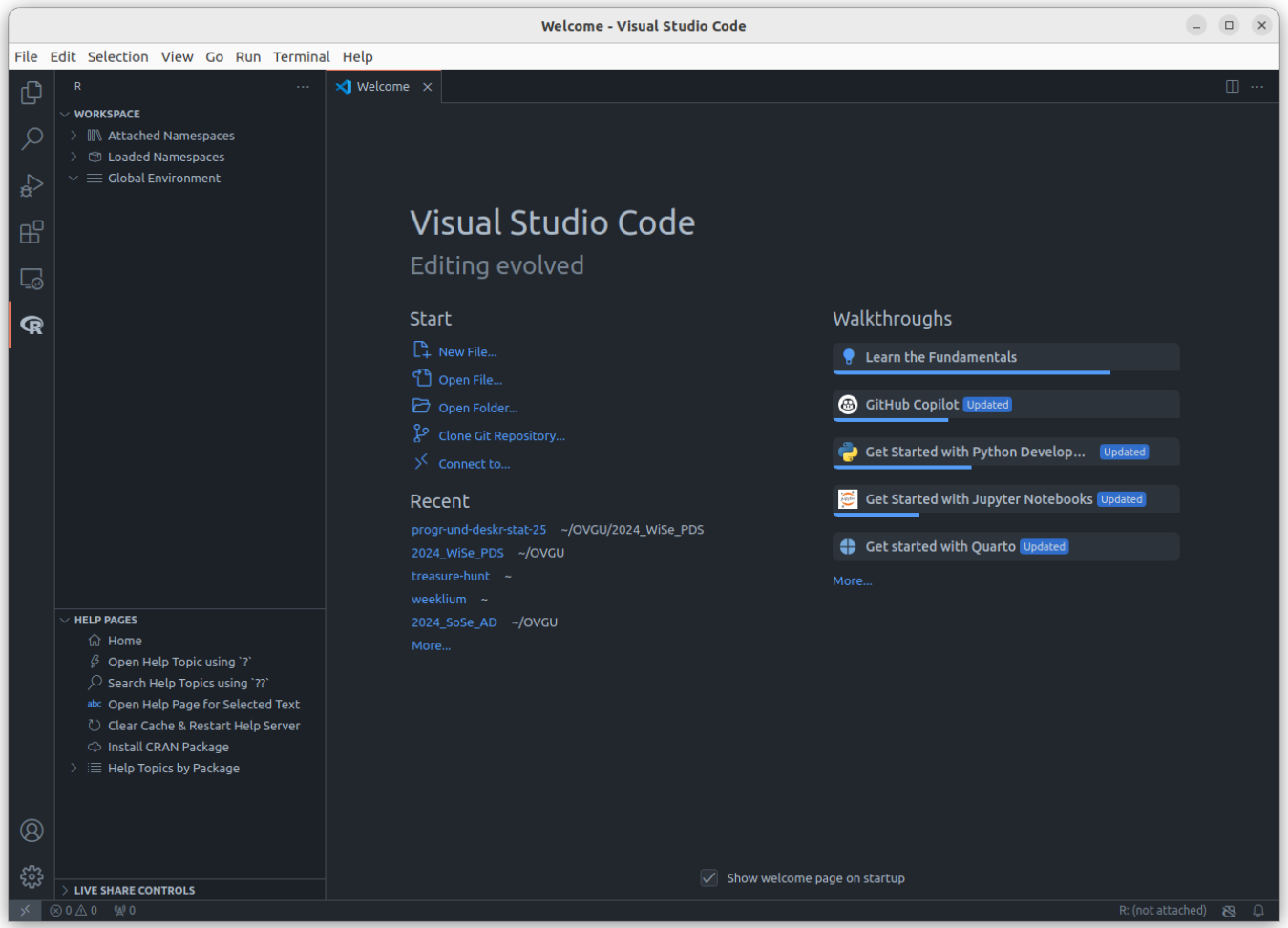
Variablen

Datenstrukturen

Installationen

- Die Programmiersprache R
- Die IDE VS Code
- VS Code für R startklar machen (Anleitung hier)

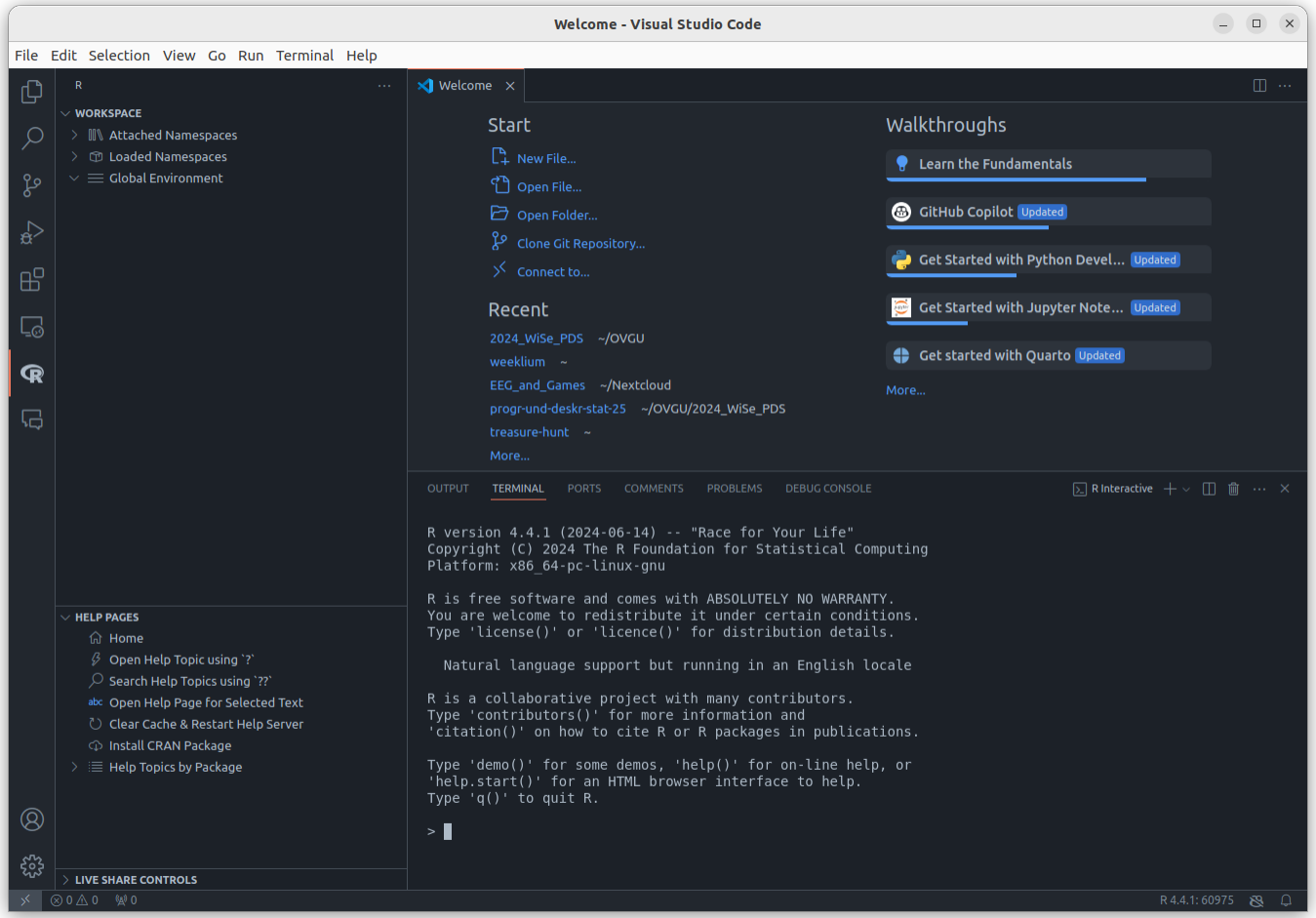
Mit VS Code und der R Extension vertraut machen



Öffnet eine R Console in VS Code

- GUI: *Terminal* → *New Terminal*
  - In die Console hinter das "$"-Zeichen "`R`" eingeben
- Keyboard Shortcut: Ctrl + Shift + P
  - In das sich öffnende Textfeld "R: Create R Terminal" eingeben

R Console in VS Code



Die Basics

- Eingabe von R Befehlen bei >
- Autocomplete mit Tab
- Code ausführen mit Enter
- Vorherige Befehle mit Cursor ↑
- Code Ausführungsstopp mit Ctrl + C

Beispiele für Befehle

```
print("Hallo Welt!")
```

```
[1] "Hallo Welt!"
```

```
sum(1:3)
```

```
[1] 6
```

Anmerkung: **Code-Snippets dieser Form immer aktiv in der Konsole nachvollziehen!**

R Funktionen

Was sind R Funktionen?

- Funktionen lassen uns etwas "tun".
- `print` und `sum` sind Beispiele für Funktionen, die etwas ausgeben bzw. aufsummieren.
- Funktionen in R werden mit runden Klammern '()' aufgerufen. In diese Klammern eingeschlossen können einer Funktion eine oder mehrere *Argumente* übergeben werden.

Hilfe zu Funktionen

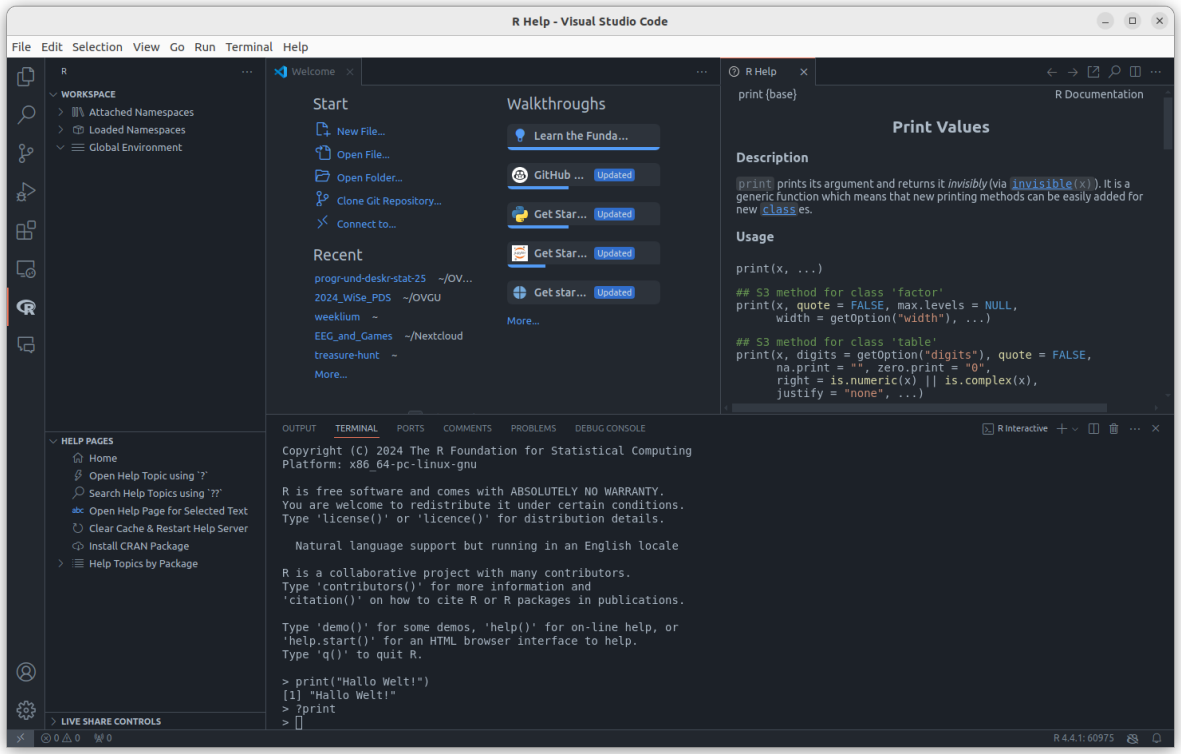
- Der einfachste und zuverlässigste Weg, Hilfe zu Funktionen zu bekommen, ist über die in der lokalen Installation integrierte Dokumentation (Help pages).
- Aufruf über Console mit Fragezeichen '?' gefolgt von dem Namen der Funktion oder mit `help(funktionsname)`.
- Das Verwenden von zwei Fragezeichen ('??') sucht nach allen Topics, die den Begriff enthalten.

```
?mean              # Öffne die Help page der Funktion "mean"
help(mean)         # Öffne die Help page der Funktion "mean"
??mean             # Suche topics, die den Begriff "mean" enthalten
```

- Die Help page jeder Funktion enthält **Description**, **Usage**, mögliche **Arguments**, **Examples** und viele weitere nützliche Informationen.
- Wie wir später noch sehen werden, gibt es weitere Ausdrücke in R, die keine Funktionen sind. In den meisten solcher Fälle finden wir die entsprechende Help page, indem der Ausdruck in Anführungszeichen gesetzt wird.

```
?"+"
?"|"
?"if"
```

Help Viewer



Mit dem Befehl `?funktionsname` oder über HELP PAGES links in der R Extension öffnen.

Beispiele

```
?print
?sqrt
```

VS Code Wiki - Interactive viewers

Neue .R Datei erstellen

- GUI: *File* → *New File* → *R Document (r)*
- Keyboard Shortcut: Ctrl + N

Bestehende .R Datei öffnen

- GUI: File → Open File
- Keyboard Shortcut: Ctrl + O

Code in einem R Skript schreiben (Befehle in Programmiersprache formulieren)

```r
print("Hallo Welt!")  # Hinter Hashtags stehen dokumentierende Kommentare
sum(1:3)              # Kommentare werden nicht ausgeführt
```

Anmerkung: **Code-Snippets dieser Form immer aktiv in einem R Skript dokumentieren!**

Befehle eines R Scripts ausführen

- Befehle an Console schicken:
  - Einzelne Zeile, auf welcher der Cursor ruht: Ctrl + Enter
  - Ausführen aller Zeilen im Skript: Ctrl + Shift + Enter
- Ausführen des gesamten R Skripts (''Source''):
  - GUI: ▷ - Symbol / Run Source
  - Keyboard Shortcut: Ctrl + Shift + S
  - in R Console:

```
source("Pfad/zum/Skript.R")
```

  - in bash Console:

```
Rscript Pfad/zum/Skript.R
```

Tipps für ein strukturiertes und übersichtliches R Skript

- Beginnen Sie ein R Skript mit einem **Header**, welcher die wichtigsten Informationen zu dem vorliegenden Programm enthält.
- Achten Sie darauf, möglichst jeden Befehl mit einem **Kommentar** zu versehen, sodass leicht nachvollzogen werden kann, was in jeder Zeile passiert.

```
# Dieses Skript gibt den Text "Hallo Welt!" aus und bildet die Summe der Zahlen von 1 bis 3.
# --------------------------
# Seminar: Programmierung und Deskriptive Statistik - WiSe 26/27
# Datum:   20-10-2026
# Autorin: Belinda Fleischmann

print("Hallo Welt!")              # Ausgabe des Texts "Hallo Welt!"
sum(1:3)                        # Summe der Zahlen von 1 bis 3
```

Das Prinzip

- Ein R Paket ist eine Sammlung von R-Funktionen, Datensätzen und vordefinierten Skripten, die in einem strukturierten Format (Paket) gebündelt sind.
- Analog zu Computerprogrammen (wie MS Word) müssen R Pakete zuerst auf dem Rechner **installiert** werden, bevor sie verwendet werden können. Ausnahmen sind Standardpakete (`standard library`), die automatisch mit der Installation von R mitgeliefert werden.
- Wie Computerprogramme müssen R Pakete auch nur *einmal* installiert werden. Deshalb empfiehlt es sich, Paketinstallationsbefehle aus Skripten auszukommentieren.
- Analog dazu, dass Computerprogramme bei jedem Neustart zuerst *geöffnet* werden müssen, bevor damit gearbeitet werden kann, müssen R Pakete bei jeder neuen R Sitzung zuerst in den Workspace **geladen** werden, bevor damit gearbeitet werden kann.

Paket-Management

- Anzeigen aller installierten Pakete in der R Extension: *HELP PAGES* → *Home* → *Packages* oder mit dem R Befehl `installed.packages()`.
- Das **Installieren** eines Pakets erfolgt mit dem R Befehl `install.packages("paketname")`. Da das Paket vor Installation noch nicht auf dem System existiert, wird der Paketname als string, d.h. **mit Anführungsstrichen** an die Funktion übergeben.
- Das **Laden** eines Pakets erfolgt mit dem R Befehl `library(paketname)`. Da das Paket nach Installation auf dem System existiert und referenziert werden kann, wird der Paketname als Variable, d.h. **ohne Anführungsstriche** an die Funktion übergeben.
- Tipp: Integrieren Sie das Paket-Management in Ihr R Skript.
  - Wenn in einem Datenanalyseskript Pakete außerhalb der ‘standard library‘ verwendet werden, sollte das Laden des Pakets auch ein Befehl im Analyseskript sein.
  - Das gilt **nicht** für Installationsbefehle (`install.packages()`)! Die Installation von Paketen ist **nur einmal** notwendig. Falls Installationsbefehle im Skript stehen, sollten sie auskommentiert werden, da sie sonst unnötig Zeit und Rechenressourcen kosten.
  - Laden Sie alle benötigten Pakete zu Beginn des Skripts und halten Sie in einem Kommentar fest, wofür es benötigt wird.

Beispiel eines strukturierten R Skripts, das zusätzliche Pakete verwendet

```r
# Dieses Skript erstellt eine character Variable und gibt deren Wert und Speicheradresse aus.
# --------------------------
# Seminar: Programmierung und Deskriptive Statistik - WiSe 26/27
# Datum:   20-10-2026
# Autorin: Belinda Fleischmann

# --- Paketemanagement ------
# install.packages("lobstr")        # Installation des Pakets "lobstr"
library(lobstr)                     # Laden des Pakets lobstr; für obj_addr()

# --- Variablendefinition ---
Ausgabetext <- "Hallo Welt!"        # Definition der Variable 'Ausgabetext'

# --- Ausgabe ---------------
print(Ausgabetext)                  # Ausgabe der Variable 'Ausgabetext'
obj_addr(Ausgabetext)               # Ausgabe der Speicheradresse der Variable 'Ausgabetext'
```

Getting started

Arithmetik, Logik und Präzedenz

Variablen

Datenstrukturen

R Konsole als Taschenrechner

```
1 + 1
```

```
[1] 2
```

```
2 * 3
```

```
[1] 6
```

```
sqrt(4)
```

```
[1] 2
```

```
exp(0)
```

```
[1] 1
```

```
log(1)
```

```
[1] 0
```

Anmerkungen:

- [1] zeigt das erste und einzige Element des Ausgabevektors an
- Vektoren werden noch im Detail behandelt.

Arithmetische Operatoren

Operator	Bedeutung
$+$	Addition
$-$	Subtraktion
$*$	Multiplikation
$/$	Division
$\hat{}$ oder $**$	Potenz
%*%	Matrixmultiplikation
%/%	Ganzzahlige Teilung (5%/%2 = 2)
%%	Modulo (5%%2 = 1)

- Matrixmultiplikation, Modulo, ganzzahlige Teilung benötigen wir zunächst nicht.
- Ganzzahlige Teilung gibt das Resultat der ganzzahligen Teilung an.
- Modulo gibt den ganzzahligen Rest bei ganzzahliger Teilung an.

Logische Operatoren

- Die Boolesche Algebra und R kennen zwei *logische Werte*: TRUE und FALSE
- Bei Auswertung von Relationsoperatoren ergeben sich logische Werte

Relationsoperator	Bedeutung
$==$	Gleich
$!=$	Ungleich
$<$, $>$	Kleiner, Größer
$<=$, $>=$	Kleiner gleich, Größer gleich
$\vert$	ODER
$\&$	UND

- $<, <=, >, >=$ werden zumeist auf numerische Werte angewendet.
- $==, !=$ werden zumeist auf beliebige Datenstrukturen angewendet.
- $\vert$ und $\&$ werden zumeist auf logische Werte angewendet.
- $\vert$ implementiert das inklusive *oder*. Die Funktion xor() implementiert das exklusive ODER.

Mathematische Funktionen

Aufruf	Bedeutung
abs(x)	Betrag
sqrt(x)	Wurzel
ceiling(x)	Aufrunden (ceiling(2.7) = 3)
floor(x)	Abrunden (floor(2.7) = 2)
round(x)	Mathematisches Runden (round(2.5) = 2)
exp(x)	Exponentialfunktion
log(x)	Logarithmusfunktion

- Hierbei handelt es sich um eine Auswahl. Einen vollständigen Überblick gibt

```
names(methods:::.BasicFunsList)
```

- R unterscheidet formal nicht zwischen Operatoren und Funktionen
- Operatoren können mit der Präfixnotation als Funktionen genutzt werden

```
`+`(2,3)            # Präfixnotation für 2 + 3
```

Operatorpräzedenz (Operatorrangfolge)

- Regeln der Form "Punktrechnung geht vor Strichrechnung".
- Vordefinierte Operatorpräzedenz kann durch Klammern überschrieben werden.

```
2 * 3 + 4
```

```
[1] 10
```

```
2 * (3 + 4)
```

```
[1] 14
```

- Generelle Empfehlung:
  - Operatorrangfolge nicht raten oder folgern, sondern nachschlagen.
  - Lieber Klammern setzen, als keine Klammern setzen.
  - Immer nachschauen, ob Berechnungen die erwarteten Ergebnisse liefern.

Präzedenz und Syntax nachschlagen

```
?Syntax  # öffnet R Help Fenster
```

Operator Syntax and Precedence

Description

Outlines `R` syntax and gives the precedence of operators.

Details

The following unary and binary operators are defined. They are listed in precedence groups, from highest to lowest.

`:: :::`	access variables in a namespace
`$ @`	component / slot extraction
`[ [[`	indexing
`^`	exponentiation (right to left)
`- +`	unary minus and plus
`:`	sequence operator
`%any% |>`	special operators (including `%%` and `%/%`)
`* /`	multiply, divide
`+ -`	(binary) add, subtract
`< > <= >= == !=`	ordering and comparison
`!`	negation
`& &&`	and
`| ||`	or
`~`	as in formulae
`-> ->>`	rightwards assignment
`<- <<-`	assignment (right to left)
`=`	assignment (right to left)
`?`	help (unary and binary)

Within an expression operators of equal precedence are evaluated from left to right except where indicated. (Note that `=` is not necessarily an operator.)

Präzedenz und Ausführungsreihenfolge arithmetischer Operatoren

Operator	Reihenfolge
^	Rechts nach links
−x,+x	Unäres Vorzeichen, links nach rechts
∗, /	Links nach rechts
+, −	Links nach rechts

Beispiele

```
2^2^3          # 2^(2^3)   = 2^8 = 256
(2^2)^3        # (2^2)^3   = 4^3 = 64
-1^2           # -(1^2)    = -1
(-1)^2         # (-1)^2    =  1
2+3/4*5        # 2+(3/4)*5 = 2+(0.75*5) = 2+3.75 = 5.75
2+3/(4*5)      # 2+3/(4*5) = 2+3/20 = 2+0.15 = 2.15
```

Getting started

Arithmetik, Logik und Präzedenz

Variablen

Datenstrukturen

Definition

In der Programmierung ist eine Variable ein abstrakter Behälter für eine Größe, welche im Verlauf eines Rechenprozesses auftritt. Im Normalfall wird eine Variable im Quelltext durch einen Namen bezeichnet und hat eine Adresse im Speicher einer Maschine. Der durch eine Variable repräsentierte Wert kann – im Unterschied zu einer Konstante – zur Laufzeit des Rechenprozesses verändert werden.

Wikipedia

- Variablen sind vom Programmierenden benannte Platzhalter für Werte
- In 3GL Sprachen wird der Variablentyp durch eine Initialisierungsanweisung festgelegt:

```
VAR A : INTEGER      # A ist eine Variable vom Typ Integer (ganze Zahl)
```

- In 3GL Sprachen wird Variablen durch eine Zuweisungsanweisung ein Wert zugeschrieben:

```
A := 1               # Der Variable A wird der numerische Wert 1 zugewiesen
```

- In 4GL Sprachen wie Matlab, Python, R werden Variablen durch Zuweisung initialisiert:

```
a = 1                # a ist eine Variable vom Typ double, ihr Wert ist 1
```

- Der Zuweisungsbefehl in Matlab und Python ist =, der Zuweisungsbefehl in R ist <- oder =.
- Offiziell empfohlen für R ist <-.

```
a <- 1               # a ist eine Variable vom Typ double, ihr Wert ist 1
a = 1                # a ist eine Variable vom Typ double, ihr Wert ist 1
```

Beispiel

Greta geht ins Schreibwarengeschäft und kauft vier Hefte, zwei Stifte und einen Füller. Wie viele analoge Gegenstände kauft Greta insgesamt?

Wir definieren zunächst alle Variablen:

```
hefte   <- 4    # Definition der Variable 'hefte'  und Wertzuweisung 4
stifte  <- 2    # Definition der Variable 'stifte' und Wertzuweisung 2
fueller <- 1    # Definition der Variable 'fueller' und Wertzuweisung 1
```

Nach Zuweisung existieren die Variablen im Arbeitsspeicher, dem sogenannten *Workspace*. Die Variablen können jetzt wie Zahlen in Berechnungen genutzt werden

```
gesamt <- hefte + stifte + fueller  # Berechnung der Gegenstandsanzahl
print(gesamt)
```

```
[1] 7
```

Ein Heft kostet einen Euro, ein Stift kostet zwei Euro, und ein Füller kostet 10 Euro. Wie viel Euro muss Greta insgesamt bezahlen?

```
gesamtpreis <- hefte * 1 + stifte * 2 + fueller * 10  # Berechnung des Preises
print(gesamtpreis)
```

```
[1] 18
```

`print()` gibt Variablenwerte in der R Konsole aus.

Der Workspace | (globaler) Arbeitsbereich

- Enthält alle aktuell definierten Objekte, Variablen und Funktionen während einer R-Sitzung.
- Nach Zuweisung werden Variablen automatisch im Workspace gespeichert.
- Ausgabe aller existierenden benutzbaren Variablen im Arbeitsspeicher in der Konsole.

```
ls()                # Anzeigen aller Variablennamen im aktuellen Workspace
```

```
[1] "fueller"     "gesamt"      "gesamtpreis" "hefte"       "stifte"
```

Löschen von Variablen

- Löschen bestimmter Variablen

```
rm(gesamtpreis)      # Löschen der Variable gesamtpreis
ls()
```

```
[1] "fueller" "gesamt"  "hefte"   "stifte"
```

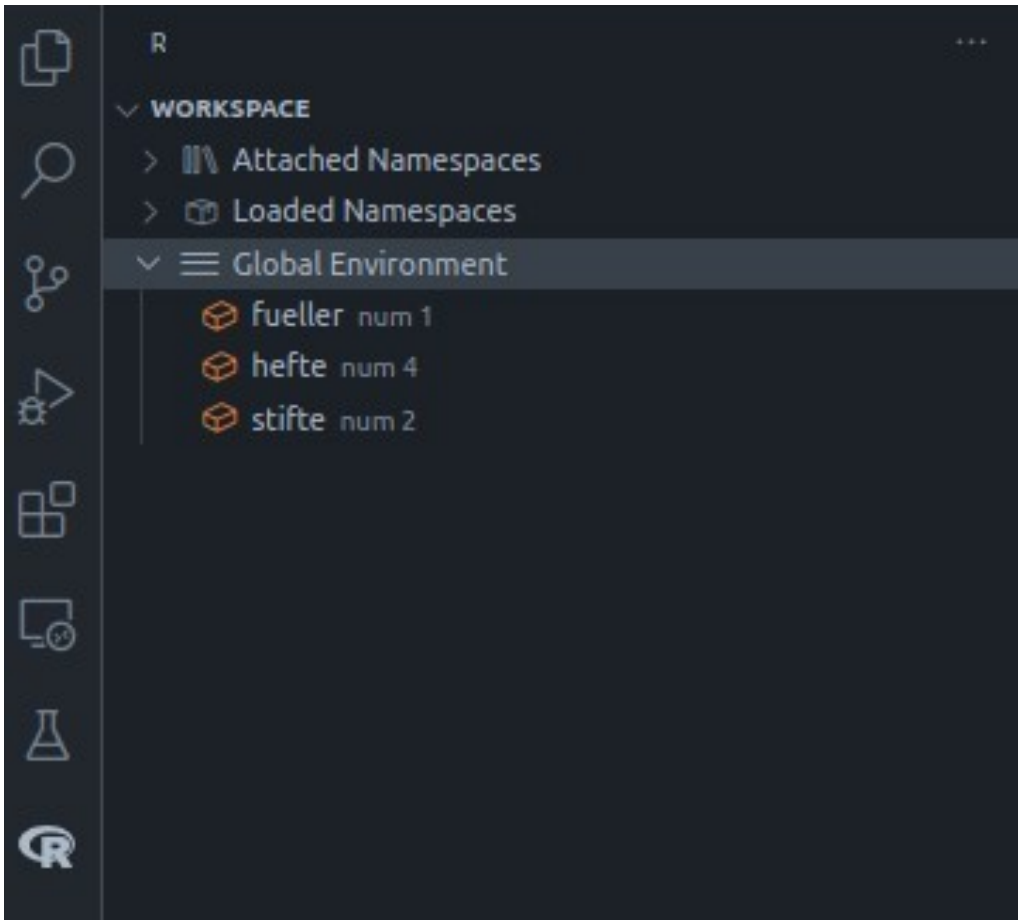
- Löschen aller Variablen im Workspace

```
rm(list = ls())      # Löschen aller Variablen
ls()
```

```
character(0)
```

Workspace

In VS Code kann der Workspace in der Seitenleiste der R Extension angezeigt werden



Zulässige Variablennamen

- bestehen aus Buchstaben, Zahlen, Punkten (.) und Unterstrichen (_).
- beginnen mit einem Buchstaben oder . nicht gefolgt von einer Zahl.
- dürfen keine *reserved words* wie for, if, NaN, usw. sein (siehe `?reserved`).
- werden unter `?make.names` beschrieben.

Sinnvolle Variablennamen

- sind kurz ($\approx$ 1 bis 9 Zeichen) und aussagekräftig.
- bestehen nur aus Kleinbuchstaben und Unterstrichen.

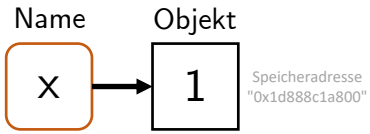
Anmerkung

- R ist *case-sensitive*. Das heißt z.B. x $\neq$ X

Variablenrepräsentation | Binding

```
x <- 1
```

- Intuitiv wird eine Variable genannt x mit dem Wert 1 erzeugt.
- De-facto geschehen zwei Dinge:
  1. R erzeugt ein Objekt (Vektor mit Wert 1) mit Speicheradresse '0x1d888c1a800'.
  2. R verbindet dieses Objekt mit dem Namen x, der das Objekt im Speicher referenziert.

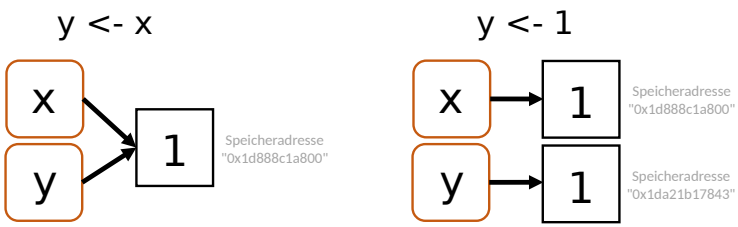



```
y <- x
```

- Intuitiv wird eine Variable genannt y mit Wert gleich dem Wert von x erzeugt.
- De-facto wird ein neuer Name y erzeugt, der dasselbe Objekt referenziert wie x.
- Das Objekt (Vektor mit Wert 1) wird nicht kopiert, R spart Arbeitsspeicher.

```
y <- 1
```

- R erzeugt ein *neues* Objekt (Vektor mit Wert 1) mit *eigener* Speicheradresse '0x1da21b17843'.
- De-facto wird ein neuer Name y erzeugt, der ein anderes Objekt referenziert als x.



Speicheradressen | object address

- Speicheradressen können mit der Funktion `lobstr::obj_addr()` angezeigt werden.

```
library(lobstr)   # Paket `lobstr` laden
x <- 1
obj_addr(x)
```

```
[1] "0x5720554f6eb8"
```

- y bekommt die Zuweisung x. De-facto referenziert y dieselbe Speicheradresse wie x.

```
y <- x
obj_addr(y)
```

```
[1] "0x5720554f6eb8"
```

- y bekommt die Zuweisung zu einem neuen Objekt (Vektor mit Wert 1). De-facto referenziert y eine andere Speicheradresse als x.

```
y <- 1
obj_addr(x)
```

```
[1] "0x5720554f6eb8"
```

```
obj_addr(y)
```

```
[1] "0x572055caae38"
```

Anmerkungen:

- Ausdrücke der Art `lobstr::obj_addr()` referenzieren eine Funktion, hier `obj_addr()`, bei gleichzeitiger Angabe des Pakets, hier `lobstr`.
- Bevor Funktionen eines Pakets verwendet werden können, muss es mit `library()` geladen werden. Ausnahmen sind R Standardpakete, die per default geladen werden, z.B. `base` oder `stats`.

Variablenrepräsentation | Copy-on-modify

```
x <- 1                          # Objekt (0x74b) erzeugt, x referenziert Speicheradresse des Objektes
y <- x                          # Der Variable y wird derselbe Wert wie x zugewiesen
obj_addr(y) == obj_addr(x)      # Zeigt, dass y dieselbe Speicheradresse (0x74b) wie x referenziert
```

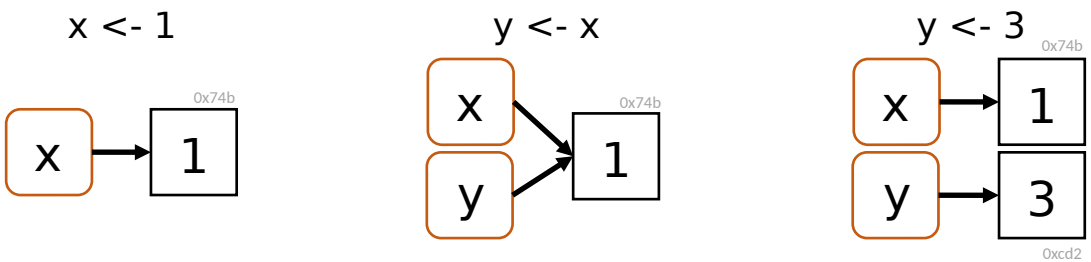
```
[1] TRUE
```

```
y <- 3                          # y modifiziert, modifizierte Kopie (0xcd2) wird gespeichert
obj_addr(y) == obj_addr(x)      # y und x referenzieren jetzt unterschiedliche Speicheradressen
```

```
[1] FALSE
```

```
y <- 1                          # Auch wenn der Wert der Zuweisung gleich ist...
obj_addr(y) == obj_addr(x)      # ...referenzieren y und x nicht dieselbe Adresse
```

```
[1] FALSE
```



- R Objekte sind *immutable*, können also nicht verändert werden.
- Bei Modifizierung der Variable y wird das „Original"-Objekt (0x74b) nicht verändert, sondern eine Kopie (0xcd2) erstellt, welche dann verändert wird.

Zur Immutability gibt es allerdings zwei Ausnahmen, genannt *Modifications-in-place*

1. Objekte mit nur einem gebundenen Namen werden in-place modifiziert

```
x <- c(1,2) # Objekt (0x74b) erzeugt, x referenziert Speicheradresse des Objektes
x[1] <- 2   # Objekt (0x74b) veraendert
```

   - Dieses Verhalten ist allerdings nur in R, nicht innerhalb VS Code reproduzierbar.

2. Environments werden in-place modifiziert.

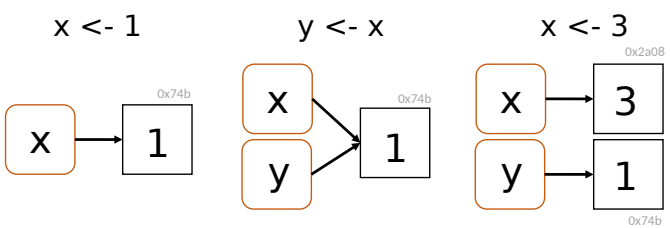
Copy-on-modify gilt auch in umgekehrter Reihenfolge

```
x <- 1                    # Objekt (0x74b) erzeugt, x referenziert Speicheradresse des Objektes
y <- x                    # Zuweisung y zu gleicher Speicherzelle wie x
obj_addr(y) == obj_addr(x) # zeigt, dass y dieselbe Speicheradresse wie x (0x74b) referenziert.
```

```
[1] TRUE
```

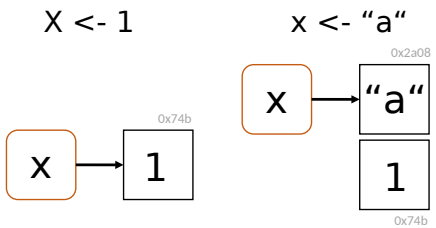
```
x <- 3                    # Ein neues Objekt mit Wert 3 wird in Speicherzelle (0x2a08) erzeugt.
                          # x referenziert jetzt Objekt (0x2a08)
                          # y referenziert weiterhin Objekt (0x74b)
obj_addr(y) == obj_addr(x) # zeigt, dass y und x nicht mehr dieselbe Speicherzelle adressieren.
```

```
[1] FALSE
```



Unbinding

```
x <- 1      #  x referenziert Objekt (0x74b)
x <- "a"    #  x referenziert Objekt (0x2a08), Objekt (0x74b) jetzt ohne Referenz
```



Garbage collection

- Nicht referenzierte Objekte im Arbeitsspeicher werden automatisch gelöscht.
- Das Löschen geschieht meist erst dann, wenn es wirklich nötig ist.
- Es ist nicht nötig, aktiv die Garbage Collection Funktion `gc()` zu benutzen.

Getting started

Arithmetik, Logik und Präzedenz

Variablen

Datenstrukturen

Klassische Datenstrukturen einer 3GL Programmiersprache

Fundamentale Datenstrukturen

- Vordefiniert innerhalb der Programmiersprache
- Logische Werte (`logical`): TRUE, FALSE
- Ganze Zahlen (`integer`): int8 (-128,…,127), int16 (–32768,…, 32767)
- Gleitkommazahlen (`single`, `double`): 1.23456, 12.3456, 123.456, …
- Zeichen (character): “a”, “b”, “c”, “!”
- Datentyp-spezifische assoziierte Operationen
  - AND, OR (logical); +, - (integer); +, -, *, / (single); Zeichenkonkatenation (character)

Zusammengesetzte Datenstrukturen

- Vordefinierte Container zur Zusammenfassung mehrerer Variablen
  (z.B. Vektoren, Listen, Arrays, Matrizen, …)
- Container-spezifische Operationen (z.B. Vektorindizierung, Matrixmultiplikation, …)

Selbstdefinierte Datenstrukturen

- Definition eigener Datenstrukturen aus vordefinierten Datenstrukturen und Containern
- Definition eigener Operationen

Datenstrukturen kennenlernen beim Erlernen einer Programmiersprache

Fundamentale Datenstrukturen

- Welche fundamentalen Datenstrukturen bietet die Sprache an?
- Welche Operationen darauf sind bereits definiert?
- Wie lautet die Syntax zur Definition einer Variable eines fundamentalen Datentyps?
- Wie lautet die Syntax, um vordefinierte Operationen aufzurufen?

Zusammengesetzte Datenstrukturen

- Welche Container und zugehörige Operationen bietet die Programmiersprache?
- Wie lautet die Syntax zum Umgang mit einem Container?

Selbstdefinierte Datenstrukturen

- Wie erzeugt man selbstdefinierte Datenstrukturen und zugehörige Operationen?
- Wie lautet die Syntax zum Umgang mit einer selbstdefinierten Datenstruktur?

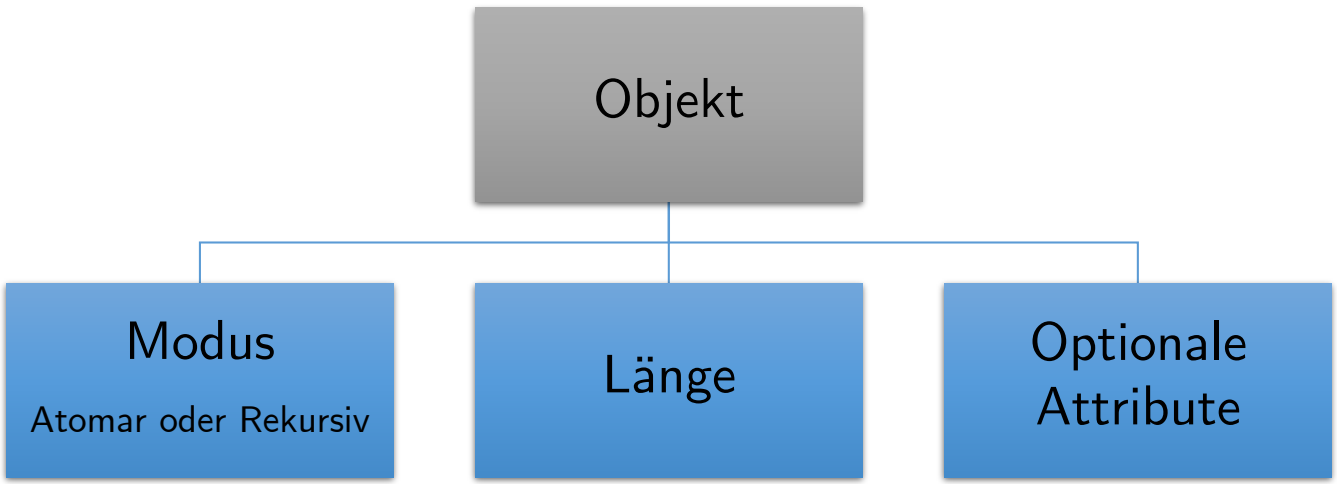
Organisation von Daten in R

Alles, was in R vorkommt, ist ein **Objekt**.

Jedem Objekt kann eindeutig zugeordnet werden:

- ein **Modus**
  - Atomar | Komponenten sind vom gleichen Datentyp.
  - Rekursiv | Komponenten können von unterschiedlichem Datentyp sein.
- eine **Länge**
- optional weitere **Attribute**

Alles, was in R vorkommt, ist ein **Objekt**.



Übersicht der R Datentypen

Datentyp	Erläuterung
logical	Die beiden logischen Werte TRUE und FALSE
double	Gleitkommazahlen
integer	Ganze Zahlen
complex	Komplexe Zahlen, hier nicht weiter besprochen
character	Zeichen und Zeichenketten (strings), 'x' oder "Hallo Welt!"
raw	Bytes, hier nicht weiter besprochen

- Double und integer werden zusammen auch als **numeric** bezeichnet.
- Viele weitere Typen, hier relevant sind **logical**, **double**, **integer**, **character**.

Übersicht der R Datentypen

Automatische Festlegung von Datentypen durch Zuweisung

```
b <- TRUE                    # logical
x <- 2.5                     # double
y <- 1L                      # (long) integer
z <- 'a'                     # character
```

Ausgabe von Datentypen durch `typeof()`

```
typeof(b)
```

```
[1] "logical"
```

```
typeof(x)
```

```
[1] "double"
```

```
typeof(y)
```

```
[1] "integer"
```

```
typeof(z)
```

```
[1] "character"
```

Übersicht der R Datentypen

Testen von Datentypen durch `is.*()`

```
is.logical(x)
```

```
[1] FALSE
```

```
is.double(x)
```

```
[1] TRUE
```

```
is.numeric(x)
```

```
[1] TRUE
```

```
is.double(y)
```

```
[1] FALSE
```

```
is.numeric(y)
```

```
[1] TRUE
```

Übersicht atomare Datenstrukturen in R

Datenstruktur	Erläuterung
Vektor	Container von indizierten Komponenten identischen Typs
Matrix	Interpretation eines Vektors als zweidimensionaler Container
Array	Interpretation eines Vektors als mehrdimensionaler Container

$\Rightarrow$ (3) Vektoren und Matrizen

Übersicht rekursive Datenstrukturen in R

Datenstruktur	Erläuterung
Liste	Container von indizierten Komponenten beliebigen Datentyps
	Insbesondere auch rekursive Struktur, z.B. Liste von Listen
Dataframe	Symbiose aus Liste und Matrix
	Jede Komponente ist Vektor beliebigen Datentyps identischer Länge

$\Rightarrow$ (4) Listen und Dataframes